Open in app ↗

Medium Search Play Games Write 3

Welcome Offer Access to everything. Now 30% off. Upgrade now

Data Engineering + Data Engineer + Databricks + Spark + Big Data +

Level 2 — Lakehouse Foundations: Delta Lake, Medallion Architecture & Ingestion Done Right

Anurag Sharma 17 min read · Sep 28, 2026



Summary

Lakehouse Foundations (Level 2) is where your Databricks workspace stops being a playground and starts being a data platform. This blog covers the three pillars everything above them depends on: Delta Lake as the transactional storage layer, the Medallion Architecture (Bronze → Silver → Gold) as the design pattern, and production-grade ingestion (Auto Loader, COPY INTO, Lakeflow Connect) as the front door. You will learn Delta's transaction log internals, schema enforcement and evolution, modern data layout with liquid clustering, table maintenance with `OPTIMIZE` / `VACUUM` and predictive optimization, time travel, and Change Data Feed — plus the mistakes that quietly break platforms at this level.

Who Should Read This Blog (Before You Dive In)

1. **Moving from a traditional Warehouse or ETL tool** — Informatica, SSIS, Talend, Teradata, Oracle. You already think in layers; you need to relearn them for object storage that isn't natively transactional.
2. **Moving from a "plain" Data Lake / Hadoop —** Spark, Hive, Parquet on S3. You write transformations fine, but your lake is probably a swamp. This blog is the fix.
3. **Junior to Mid-Level Data Engineer on Databricks —** You can create tables and run Auto Loader from a copied snippet, but you don't know what a checkpoint really is, why your table has 40,000 tiny files, or when partitioning helps versus hurts.
4. **Preparing for a Databricks or Data Engineering interview —** Medallion architecture, Delta internals, schema evolution, and "why is my query slow?" are Level 2 questions asked in nearly every interview, including senior ones.
5. **Tech Lead diagnosing a messy lake —** Read Steps 6–8 first. That's where the cost and reliability damage usually lives.

The Day the "Data Lake" Became a Swamp

There's a pattern I have seen in almost every organization that adopted a data lake too early.

Year one: excitement. Someone spins up S3 or ADLS, dumps everything in — CSVs, JSON logs, exports, Parquet files with names like `final_v2_FINAL_use_this.csv`. A few notebooks read it. Everyone is happy.

Year two: hesitation. Analysts stop trusting the numbers. Two dashboards show two different revenue figures. Nobody knows which file is the source of truth. A `DELETE` statement wipes the wrong partition. Someone re-runs a job and doubles the row count.

Year three: the Swamp. Nobody wants to touch the lake. Data engineering becomes a ticket queue of manual fixes. The warehouse wins by default because at least it's *consistent*.

The lakehouse exists precisely to fix that failure mode. It keeps the openness, scale, and cheap object storage of a data lake — and adds the things a warehouse always had: transactions, schema, governance, and reliability.

Why Lakehouse Foundations (Level 2) Is the Most Underrated Level

Databricks Workspace (Level 1) taught you to drive the workspace. Level 3 onwards is about building on top of storage.

Lakehouse Foundations (Level 2) is the storage and ingestion layer — and every single level above it inherits the consequences of what you do here:

Decision you make at Level 2	How it punishes you later
Bad medallion boundaries	Level 3 transformations become spaghetti; nobody knows where to put logic
Small files from streaming ingestion	Level 6 optimization becomes a full-time job; query costs explode
No schema enforcement	Silent data corruption reaches executives before anyone notices
Checkpoint misconfiguration	Duplicate data on every pipeline restart — discovered months later
Random partitions on high-cardinality columns	Data skipping fails; every query scans everything
No retention/vacuum strategy	Storage bill grows forever; time travel breaks unexpectedly
No ingestion-time metadata (`_ingest_time` , `_source_file`)	You can never answer "when did this row arrive?" or replay cleanly

Most “Databricks performance problems” I have been asked to debug were actually **Lakehouse Foundations (Level 2) decisions made six months earlier.**

Step 1: The Lakehouse Idea — What Problem Lakehouse Foundation Solves

Two systems dominated the last two decades:

- **Data Warehouse:** Consistent, fast, governed — but expensive, rigid, and poor at semi-structured data, streaming, and ML.
- **Data Lake:** Cheap, flexible, open — but no transactions, no schema guarantee, no governance. The swamp.

The lakehouse argues you don’t need both. Put open file formats on cheap object storage and add a transactional metadata layer on top. That metadata layer is what turns a folder of Parquet files into a *table*.

On Databricks, that transactional layer is **Delta Lake** — and in 2026 the platform default is **Unity Catalog managed tables using Delta Lake or Apache Iceberg**, with `INSERT`, `UPDATE`, `DELETE`, and `MERGE` all supported through the same table interface.

The mental model to lock in:

```
Object storage (S3 / ADLS / GCS)  ← cheap, durable, dumb
        +
Delta Lake transaction log         ← smart, ACID, this is the table
        =
A table that behaves like a database table
```

Interview framing: "A Delta table isn't the Parquet files. The Parquet files are the *data*; the transaction log is the *table*." Say that, and you sound like you have actually operated one.

Step 2: Delta Lake Internals — The Transaction Log Is the Whole Story

When you write to a Delta table, two things happen:

1. **Data files** are written as Parquet into the table directory.
2. **A commit** is atomically recorded as a JSON file in `_delta_log/`.

```
my_delta_table/
├── part-00000-xxxx.snappy.parquet     ← data
├── part-00001-xxxx.snappy.parquet
├── part-00002-xxxx.snappy.parquet
└── _delta_log/
```

```
├── 00000000000000000000.json      ← commit 0: table created
├── 00000000000000000001.json      ← commit 1: files added
├── 00000000000000000002.json      ← commit 2: files removed (UPDATE)
├── ...
├── 00000000000000000010.checkpoint.parquet   ← every 10 commits
├── 00000000000000000020.checkpoint.parquet
└── _last_checkpoint               ← pointer for fast recovery
```

Each JSON commit contains **actions**: `add` (new files), `remove` (files logically deleted), `metaData` (schema/config changes), and `txn` (transaction identifiers).

Why this design matters in practice:

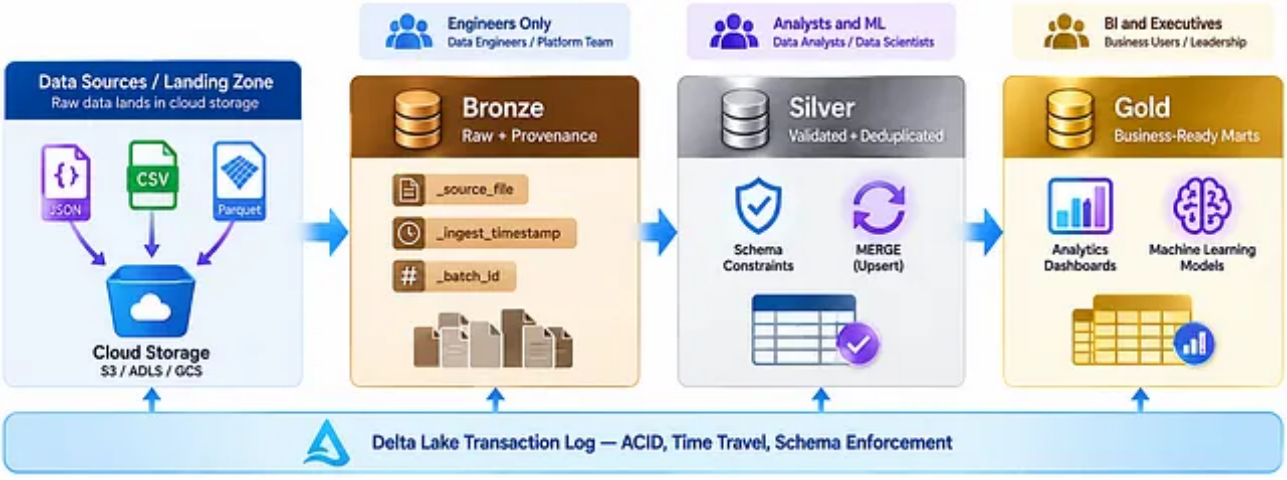
- **ACID on object storage.** Object stores don't offer multi-file atomicity. Delta fakes it by writing data files first, then committing a single JSON file. If the job dies mid-write, the commit never happens — orphan files exist but the table never sees them.
- **Snapshot isolation.** Readers see a consistent version even while writers commit. This is why concurrent reads and writes don't corrupt your table.
- **Optimistic concurrency control.** Two writers try to commit at once; one wins, the other retries and fails cleanly with a `ConcurrentAppendException` / `ConcurrentModificationException` — *not* corrupt data.
- **Checkpoints make it fast.** Replaying thousands of JSON files to read a table would be slow, so Delta periodically writes a Parquet checkpoint — every 10 commits by default. Delete `_delta_log` and your data files become meaningless noise. Never touch data files directly — Databricks explicitly warns that manual modification causes lost data or corruption.
- **Feature flags live in `metaData`.** Deletion vectors, column mapping, liquid clustering keys, retention properties — all recorded here. This is why you can change table *layout* without rewriting history.

Things you should never do manually (Delta handles it): `REFRESH TABLE`, `ALTER TABLE ADD/DROP PARTITION`, `MSCK REPAIR`, and reading partitions by path (`/date=2026-01-01`). Use `WHERE` clauses and let data skipping work.

Mastery checkpoint: You can explain what happens to `_delta_log` during a failed write, and why `MERGE` doesn't corrupt a table when two jobs overlap.

Step 3: The Medallion Architecture — Bronze, Silver, Gold

The medallion pattern is the most widely adopted data organization in the Databricks world. It works because it separates **fidelity** from **quality** from **purpose**.



Layer	Purpose	Data state	Who consumes it	Write pattern
Bronze	Faithful landing / archive	Raw, exactly as received + ingestion metadata	Data engineers only	Append-only, re-runnable
Silver	Trusted, conformed, deduplicated	Validated, typed, PII handled, one row = one entity	Analysts, ML engineers	MERGE / upsert (SCD)
Gold	Business-ready marts	Aggregated, modeled, purpose-built	BI, executives, downstream apps	Overwrite per partition / materialized

Rules that make medallion work (and that most teams violate):

1. **Bronze holds raw data with *metadata columns*, not transformations.**

```
-- Every bronze table should carry provenance
SELECT
  *,
  _source_file,          -- which file/object delivered this row
  _ingest_timestamp,     -- when we landed it
  _batch_id,             -- which pipeline run brought it
  _pipeline_run_id
FROM bronze.raw_orders;
```

Without these, you can never answer “when did this arrive?”, “was this row processed twice?”, or “re-ingest only this batch.”

2. **Never fix data in Bronze.** The moment you clean data in Bronze, you lose your ability to replay. Bronze is the audit copy you will be grateful for the first time an upstream system sends corrupt data for three months.

3. **Silver is the “single source of truth.”** Cleansing, deduplication, type casting, PII masking, and business entity resolution belong here. Nothing downstream should be reconciling duplicates.

4. **Gold optimizes for consumption, not for storage elegance.** Denormalized is fine. Wide is fine. Star schemas with fact and dimension tables are fine.

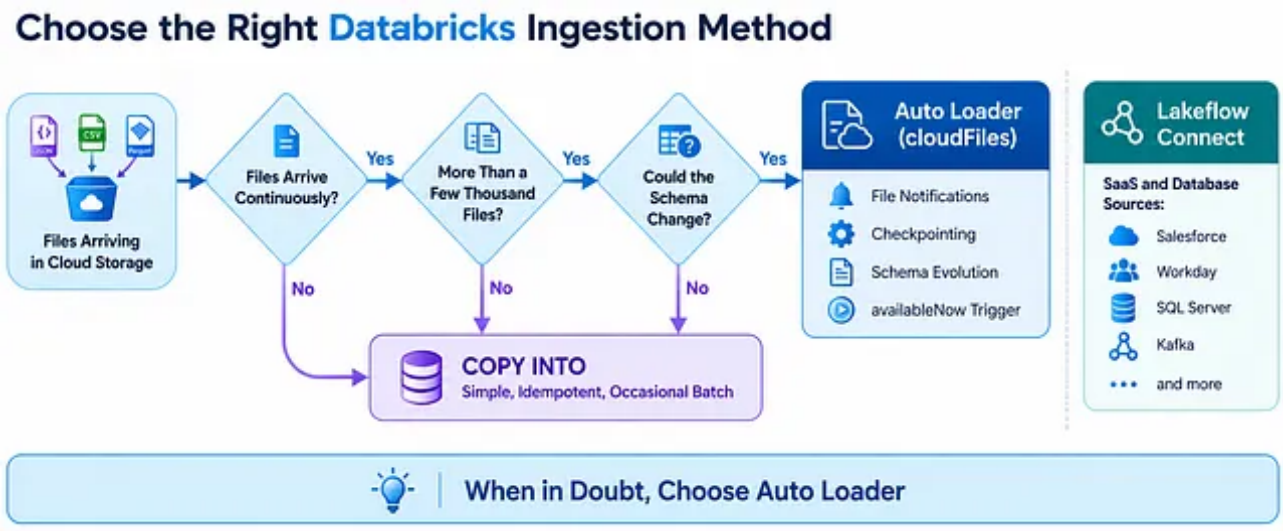
5. **Bronze and Silver are re-derivable; Gold should be cheap to rebuild.** If your Gold layer takes six hours to rebuild, that’s a Level 3/4 problem, not a medallion failure.

The anti-pattern I see most: teams implement four, five, or seven layers (“Raw → Staging → Cleansed → Curated → Trusted → Semantic → Reporting”) because each team adds its own. The result is a layer nobody owns and a 40-hour pipeline. Three layers, clearly owned. If you need a fourth, justify it explicitly.

A subtle 2026 point: with **Unity Catalog managed tables**, your catalog/schema naming can *be* your medallion — e.g. `prod_catalog.bronze_orders`, `prod_catalog.silver_orders`, `prod_catalog.gold_orders`. This makes lineage and access control dramatically simpler than scattering layers across workspaces.

Step 4: Ingestion — Auto Loader, COPY INTO, and Lakeflow Connect

This is the front door. Get it wrong and everything downstream inherits the damage.



Path	Best for	Key behaviors
Auto Loader (`cloudFiles`)	Continuous or high-volume file arrivals, thousands to millions of files, evolving schemas	Incremental, checkpoint-based, schema inference + evolution, file notification mode available
COPY INTO	Ad-hoc, simple, occasional batch loads	Idempotent by design, tracks already-loaded files, no checkpoint management
Lakeflow Connect / managed connectors	SaaS and database sources (Salesforce, Workday, SQL Server, Kafka, etc.)	Managed pipelines, less code, governance-aware

Decision rule from Databricks' own guidance: ask three questions — *Do files arrive continuously? Will the file count exceed a few thousand? Could the schema change?* If any answer is yes, use Auto Loader. If all are no, `COPY INTO` is fine. **When in doubt, pick Auto Loader** — it still fits when the workload grows.

Production Auto Loader pattern (batch-triggered, cost-efficient):

```
# Bronze ingestion: idempotent, schema-evolving, replayable
schema_loc = "abfss://meta@storage/auto_loader/orders/schema"
checkpoint  = "abfss://meta@storage/auto_loader/orders/checkpoint"

(spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", schema_loc)
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")   # default
    .option("cloudFiles.useNotifications", "true")               # avoids LIST A
    .option("cloudFiles.maxFilesPerTrigger", "1000")
    .option("cloudFiles.includeExistingFiles", "true")
    .load("abfss://landing@storage/orders/")
    .select(
        "*",
        "_metadata.file_path".alias("_source_file"),
        "_metadata.file_modification_time".alias("_file_mod_time"),
        current_timestamp().alias("_ingest_timestamp")
    )
    .writeStream
    .format("delta")
    .option("checkpointLocation", checkpoint)
    .option("mergeSchema", "true")     # required when addNewColumns adds column
    .trigger(availableNow=True)        # process what's there, then stop — cheap
    .toTable("prod_catalog.bronze_orders")
)
```

The four things that decide whether this is reliable:

1. **`checkpointLocation` is not optional.** The checkpoint records which files have already been processed. Lose it and you either reprocess everything

or silently skip files. Data loss from checkpoint misconfiguration is one of the hardest production bugs to diagnose.

2. **Use file notification mode for scale.** `useNotifications=true` avoids repeatedly listing the whole directory — a hidden but enormous cloud API cost at scale.
3. **`availableNow=True` is the modern default for scheduled pipelines.** It ingests everything available and stops, instead of running a cluster 24/7. Continuous mode is for genuinely low-latency needs only.
4. **Idempotency is a design property, not a hope.** `COPY INTO` guarantees it. Auto Loader + checkpointing + `MERGE` in Silver achieves it. Blind `append` in Silver does not.

COPY INTO for simple, safe loads:

```
COPY INTO prod_catalog.bronze_customers
FROM 'abfss://landing@storage/customers/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
```

Re-run it ten times — you still get one copy of each file. That idempotency is why it survives audits so well.

Step 5: Schema Management — Enforcement, Evolution, and the Rescue Column

Schema is where lakehouses are won or lost. Databricks gives you both a **seatbelt** and an **escape hatch** — knowing which to use when is a Level 2

differentiator.

Enforcement (the seatbelt):

```
CREATE TABLE prod_catalog.silver_orders (
    order_id     STRING NOT NULL,
    customer_id  STRING NOT NULL,
    order_ts     TIMESTAMP,
    amount       DECIMAL(12,2),
    status       STRING,
    CONSTRAINT valid_amount CHECK (amount >= 0),
    CONSTRAINT valid_status CHECK (status IN ('NEW','PAID','SHIPPED','CANCELLED'
)
USING DELTA;
```

- A write with a mismatched schema fails rather than corrupting the table.
- `NOT NULL` and `CHECK` constraints are enforced on new writes.
- This is your first line of defense against upstream “quick fixes.”

Evolution (the escape hatch):

```
# Streaming / batch write that tolerates new columns
df.write.option("mergeSchema", "true").mode("append").saveAsTable("prod_catalog.
```

Auto Loader schema evolution modes:

Mode	Behavior	Use when
`addNewColumns` (default)	Adds new columns to the schema automatically	Most ingestion pipelines
`rescue`	New columns land in `_rescued_data` instead of altering schema	You want a human decision before schema changes
`failOnNewColumns`	Pipeline fails and waits for manual intervention	Strictly governed Bronze feeds
`none`	Ignores new columns entirely	Fixed-schema feeds (rare, dangerous)

The `_rescued_data` column is one of the most underrated features in Databricks. When data doesn't match the declared schema — bad type, unexpected field, malformed record — it isn't dropped; it's parked in `_rescued_data` as JSON.

```
-- Find records your schema rejected
SELECT _source_file, _rescued_data
FROM prod_catalog.bronze_events
WHERE _rescued_data IS NOT NULL
LIMIT 50;
```

Interview gold: "What happens when a new column appears upstream?" A weak answer is "the job fails." A strong answer covers default `addNewColumns` evolution, the `mergeSchema` write requirement, `_rescued_data` for schema mismatch rather than drift, and the trade-off between automatic evolution (velocity) and controlled evolution (governance).

Step 6: Data Layout — Why Partitioning Is Usually the Wrong Answer Now

This is the single biggest change in Databricks best practice over the last few years, and many teams — and many blogs, and many interview answer keys — are still stuck in 2019.

The old advice: partition by date, then `ZORDER BY` your filters. Modern reality: Databricks now recommends **liquid clustering for all new tables**, including streaming tables and materialized views. Current guidance is explicit that *most tables with less than 100 TB of data don't need partitioning at all*, and that partitioning "likely negatively affects performance more often than it helps."



Why the change?

- Partitioning works only for **low or known cardinality** columns (date, region) — high-cardinality columns (`customer_id` , raw timestamps) create the *small file problem* immediately.
- Changing partition keys means **rebuilding the table.**
- You **cannot** `ZORDER` on partition columns, and Z-Ordering can only reorder **within a** partition, not across the table.
- `ZORDER` re-sorts the whole table on every `OPTIMIZE` ; the work never shrinks as the table grows.

Liquid clustering removes these constraints:

```
-- Recommended for all new tables
CREATE TABLE prod_catalog.silver_orders (
    order_id     STRING,
    customer_id  STRING,
    order_ts     TIMESTAMP,
    amount       DECIMAL(12,2)
```

```
)
CLUSTER BY (customer_id, order_ts);
```

- No partitions required; no cardinality concern.
- Change keys any time with a single `ALTER TABLE ... CLUSTER BY (...)`. Metadata update only — the next `OPTIMIZE` re-clusters **incrementally**, so cost scales with *new* data, not table size.
- Works for both low- and high-cardinality columns and avoids fixed partition boundaries.
- If queries group or filter heavily on one column, prioritize clustering keys — but prioritizing a high-cardinality column can reduce effectiveness of other keys.

Converting existing tables:

```
-- Unpartitioned existing table
ALTER TABLE prod_catalog.silver_orders CLUSTER BY (customer_id, order_ts);

-- On DBR 18.1+, convert a partitioned table in place
ALTER TABLE prod_catalog.silver_orders
REPLACE PARTITIONED BY WITH CLUSTER BY AUTO;   -- or name your keys explicitly
```

Migration mapping Databricks recommends:

Your current technique	Recommended clustering keys
Hive-style partitioning	Previously used partition columns
Z-order indexing	Previously Z-ordered columns
Both	Partition columns + Z-ordered columns
Generated columns to reduce cardinality	The original column (skip the generated column)

One important nuance: For unpartitioned tables, Delta **automatically applies ingestion-time clustering** — you get date-style query performance without manual tuning. But heavy `UPDATE` / `MERGE` activity can erode that; Databricks recommends liquid clustering on a column aligned with ingestion order (event timestamp, creation date) in those cases.

When *should* you still partition? Roughly: 100 TB+ tables, after validating liquid clustering isn't enough, with each partition holding **at least 1 GB** of data. Otherwise, don't.

Mastery checkpoint: If someone on your team asks "which column should we partition by?", the correct first answer in 2026 is "probably none — what's your query pattern, and let's cluster instead."

Step 7: Table Maintenance — OPTIMIZE, VACUUM, and Predictive Optimization

Streaming ingestion writes small files. Small files destroy query performance and inflate cloud API costs. Maintenance is not optional at Level 2 — it's part of the design.

`OPTIMIZE` — compaction:

```
-- Compact small files
OPTIMIZE prod_catalog.silver_orders;

-- Legacy: Z-order (use liquid clustering for new tables instead)
OPTIMIZE prod_catalog.silver_orders ZORDER BY (customer_id);
```

`OPTIMIZE` rewrites small files into larger, better-laid-out ones. Databricks automatically tunes target file sizes based on table size. **It does not delete old files,** so storage grows until you vacuum.

`VACUUM` — reclaim storage:

```
VACUUM prod_catalog.silver_orders;              -- default 7-day retention
VACUUM prod_catalog.silver_orders RETAIN 168 HOURS;
```

Two rules that matter:

- Default retention is **7 days.** Vacuuming more aggressively requires disabling the safety check and lowers your time-travel window — and **breaks any reader using an older snapshot.**
- If a long-running job or Delta Sharing consumer is reading a version older than your retention, vacuuming removes files out from under them. **Set retention to match your longest reader, not your shortest.**

Predictive Optimization is the modern answer for Unity Catalog managed tables: Databricks automatically runs `OPTIMIZE` and `VACUUM` based on usage patterns. Enable it and stop hand-scheduling maintenance jobs. Combined with liquid clustering, this is the current recommended default stack.

The small file problem, concretely: a streaming pipeline writing every minute for a year creates ~525,600 files. A query that must open metadata for all of them loses more time to file listing than to actual data processing. This is the single most common reason a working job suddenly becomes “slow” three months after launch.

Also from Databricks' Delta best practices — things people get wrong:

- **Don't use Spark caching with Delta.** You lose data skipping from filters applied on top of the cached DataFrame, and the cache can go stale because Delta doesn't always know the same table was modified via a different identifier.
- **Don't manually `REFRESH TABLE`, add/drop partitions, or run `MSCK`.**
- **Don't manually edit Parquet files** in a Delta table. Ever.
- **Remove legacy Delta/Spark configs** when upgrading runtimes — stale configs block new optimizations and defaults from taking effect. This is a silent performance killer during migrations.

Step 8: Time Travel and Change Data Feed

Two Delta features that turn "we think this changed" into "here's exactly what changed."

Time travel — query any previous version:

```
-- By version
SELECT * FROM prod_catalog.silver_orders VERSION AS OF 42;

-- By timestamp
SELECT * FROM prod_catalog.silver_orders TIMESTAMP AS OF '2026-08-01T00:00:00';

-- Inspect history
DESCRIBE HISTORY prod_catalog.silver_orders;

-- Restore a table to a prior version (use with care, and audit it)
RESTORE TABLE prod_catalog.silver_orders TO VERSION AS OF 42;
```

Time travel is your incident-response tool: *"what did this table look like before yesterday's bad deployment?"* It is bounded by `VACUUM` retention. It is not a backup strategy — for genuine disaster recovery you still need cross-region copies.

Change Data Feed (CDF) — the delta between versions:

```
ALTER TABLE prod_catalog.silver_orders
SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

```
-- Row-level changes: insert / update_preimage / update_postimage / delete
SELECT *
FROM table_changes('prod_catalog.silver_orders', '2026-09-01', '2026-09-12')
WHERE _change_type <> 'update_preimage';
```

```
# Incremental downstream processing
(spark.readStream
    .option("readChangeFeed", "true")
    .option("startingVersion", 0)
    .table("prod_catalog.silver_orders")
    .writeStream
    .option("checkpointLocation", "/checkpoints/orders_cdf")
    .toTable("prod_catalog.gold_orders_incremental")
)
```

Why CDF matters at Level 2: it's how Silver feeds Gold *incrementally* instead of doing full reloads. It also produces the audit trail that Level 7 governance and Level 3 SCD Type 2 logic depend on. Enable it deliberately — it adds storage and write overhead — typically on Silver and above, not on high-volume Bronze.

Step 9: Governance Touchpoints at Lakehouse

You don't need full governance at Lakehouse, but the *seams* must be right — retrofitting governance onto a platform built without them is painful.

Do these at Lakehouse:

- Use Unity Catalog managed tables rather than raw paths like `dbfs:/mnt/...`. Managed tables get automatic storage location management, predictive optimization, fine-grained access control, and lineage.
- Govern external storage with external locations, not ad-hoc mounts. Mounts are the legacy pattern that makes Governance, Sharing & AI Scale significantly harder.
- Keep secrets in secret scopes, never in notebooks.
- Tag tables at creation — `team`, `domain`, `sensitivity`, `environment`. Tags drive cost chargeback and policy enforcement. Retrofitting tags across 4,000 tables is a project nobody wants.
- Name deterministically. `{env}_{layer}_{domain}_{entity}` in a catalog/schema that expresses ownership. Your future self searching for "the orders table" will thank you.
- Tag and isolate PII from day one. Don't land PII in Silver in plain text assuming you'll mask it later — later never comes.

Step 10: Declarative Pipelines — A First Look at Streaming Tables & Materialized Views

Historically, Lakehouse was code-first: you wrote Auto Loader reads and `MERGE` statements by hand. Today there's a second, declarative option worth knowing early.

Lakeflow Declarative Pipelines (formerly Delta Live Tables) let you *declare* what a table should be, and the pipeline figures out ordering, incremental processing, retries, and dependencies:

```
import dlt
from pyspark.sql.functions import col, current_timestamp

@dlt.table(name="bronze_orders", comment="Raw orders with ingestion metadata")
def bronze_orders():
    return (spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.schemaLocation", "/meta/bronze_orders")
        .load("abfss://landing@storage/orders/")
        .withColumn("_ingest_timestamp", current_timestamp()))

@dlt.table(name="silver_orders", comment="Validated orders")
@dlt.expect_or_drop("valid_order_id", "order_id IS NOT NULL")
@dlt.expect_or_fail("non_negative_amount", "amount >= 0")
def silver_orders():
    return (dlt.read_stream("bronze_orders")
        .filter(col("status").isNotNull())
        .dropDuplicates(["order_id"]))

@dlt.table(name="gold_orders_by_region", comment="Daily revenue by region")
def gold_orders_by_region():
    return (dlt.read("silver_orders")
        .groupBy("region")
        .agg({"amount": "sum"}))
```

Three reasons this belongs in a Lakehouse conversation even though pipelines are Pipelines Engineering:

1. **Streaming tables and materialized views live in this layer conceptually** — they're storage objects with freshness guarantees, not just

orchestration.

2. **Expectations are data quality at the ingestion boundary —** `expect_or_drop`, `expect_or_fail`, `expect` let you define quality rules alongside the table definition instead of bolting on checks later.
3. **Liquid clustering applies to streaming tables and materialized views,** so layout decisions are made here too.

When to choose which: declarative pipelines for standard medallion ingestion where you want managed reliability and less code; hand-written Structured Streaming when you need maximum control or unusual logic. Both are legitimate — just make the choice explicit rather than accidental.

Step 11: Interview Questions You Will Actually Be Asked at This Level

#	Question	What a strong answer includes
1	What makes Delta Lake ACID on top of object storage?	`_delta_log` JSON commits, atomic single-file commit, snapshot isolation, optimistic concurrency, periodic checkpoints
2	Explain the medallion architecture and why not just two layers.	Bronze = raw + provenance + replayability; Silver = deduplicated/conformed single source of truth; Gold = consumption-optimized. Two layers conflates lineage with trust.
3	Auto Loader or COPY INTO — how do you choose?	Continuity, file volume (~a few thousand), schema drift → Auto Loader; simple occasional batch → COPY INTO; default to Auto Loader
4	What is a checkpoint and what breaks without it?	Records processed files/offsets; without it you reprocess everything (duplicates) or skip files (data loss)
5	How do you handle schema evolution?	`mergeSchema`, Auto Loader `schemaEvolutionMode`, `_rescued_data`, controlled vs automatic evolution trade-off
6	Would you partition this table?	Ask about query patterns and volume; most tables under 100 TB don't need it; recommend liquid clustering; partition only with ≥ 1 GB per partition
7	Partitioning vs Z-ORDER vs liquid clustering.	Cardinality limits, table rebuild requirement, incremental vs full re-clustering, key-change capability
8	Your streaming table has 200k small files. Fix it.	Diagnose ingestion frequency + trigger config; `OPTIMIZE` (incremental with liquid clustering); predictive optimization; align target file size; adjust trigger/ `maxFilesPerTrigger`
9	How do you reclaim storage without breaking readers?	`VACUUM` with retention ≥ longest reader's time-travel window; understand default 7 days; check Delta Sharing consumers
10	How do you process only changed rows downstream?	Change Data Feed, `table_changes()`, `readChangeFeed` streaming, SCD Type 2 in Silver
11	How do you recover from a bad write?	`DESCRIBE HISTORY`, time travel to prior version, `RESTORE`, then fix upstream; note retention limits
12	Why does Databricks recommend against Spark `cache()` with Delta?	Lost data skipping on post-cache filters; stale cache when table is modified through another identifier
13	Design ingestion for a source that adds columns unpredictably.	Auto Loader + notifications + `rescue` mode to raw Bronze, `_rescued_data` monitoring, curated promotion to Silver with explicit schema
14	How do you guarantee an ingestion job is idempotent?	Idempotent source tracking (COPY INTO / checkpoints), `MERGE` on stable keys, `_batch_id` for traceability and safe replay

Step 12: Hands-On Lab — Build Level 2 End to End

Do this once at small scale, and Lakehouse stops being theory.

1. **Land raw files.** Copy a few JSON/CSV files into a governed volume or external location.
2. **Create Bronze with Auto Loader** using file notification mode, `availableNow=True`, and `_source_file` / `_ingest_timestamp` metadata. Write to a Unity Catalog managed table.
3. **Verify idempotency.** Drop a duplicate file into the landing folder. Confirm it isn't double-ingested. Then delete the checkpoint in a *sandbox* table and observe the failure mode — understand it, don't fear it.
4. **Promote to Silver with MERGE** on a business key, with `NOT NULL` and `CHECK` constraints, deduplication, and type standardization.
5. **Break the schema on purpose.** Add a new field upstream. Observe `addNewColumns` behavior; then switch a sandbox to `rescue` mode and query `_rescued_data`.
6. **Ingest 100 tiny files** to manufacture the small file problem. Run `OPTIMIZE` and measure the before/after file count and query time.
7. **Switch to liquid clustering.** Run `ALTER TABLE ... CLUSTER BY (key)`. Compare query performance against the partitioned version.
8. **Enable CDF and build Gold incrementally** with a streaming read of the change feed.
9. **Time travel.** Run `DESCRIBE HISTORY`, query an older version, deliberately make a bad update, and revert it with `RESTORE`.
10. **Run maintenance.** `OPTIMIZE`, then `VACUUM` with a retention you can justify, and confirm older versions are gone.
11. **Turn on predictive optimization** and remove your hand-scheduled maintenance job. Watch what happens.
12. **Document your layout decisions in a README:** why these clustering keys, this retention, these layers. That document *is* your interview story.

Step 13: Common Mistakes at Level 2 (and the Fix)

Mistake	Symptom	Fix
Transforming data in Bronze	Can't replay or audit; lineage lost	Keep Bronze raw + metadata; transform in Silver
Partitioning by high-cardinality columns	Thousands of tiny directories, slow queries	Liquid clustering instead
Z-ORDER on partitioned tables	Optimization barely helps; can't cross partition boundaries	Cluster, don't partition + Z-order
No `OPTIMIZE` on streaming tables	Query performance degrades steadily over months	Predictive optimization or scheduled `OPTIMIZE`
`VACUUM` with 1-hour retention	Time travel and long readers break	Retention ≥ longest consumer; 7-day default is a floor for a reason
Spark `cache()` on Delta tables	Stale or partially-skipped results	Let Delta handle it; don't cache
Mounts everywhere instead of external locations	Level 7 governance becomes a rewrite	Use Unity Catalog external locations
`append` into Silver "for now"	Duplicates discovered quarters later	`MERGE` on stable keys from day one
No `_batch_id` / `_ingest_timestamp`	Can't reconcile "when did this arrive?" or replay a batch	Add provenance columns at ingestion, always
Multiple unexplained intermediate layers	40-hour pipeline, no clear ownership	Three layers, one owner each
Manual `REFRESH TABLE` and partition `ALTER`s	Fragile, unnecessary, occasionally corrupting	Delete them from your codebase
Legacy configs carried into new runtimes	New optimizations silently disabled	Audit and remove legacy Delta/Spark properties

Conclusion

Level 2 is where your platform's character is decided.

You now have the complete foundation:

- **Delta Lake** gives you ACID transactions, schema guarantees, time travel, and change data capture on cheap object storage — and the transaction log is the table.
- **The Medallion Architecture** gives every dataset a defined purpose, owner, and trust level, so nobody has to guess whether a number is right.
- **Modern ingestion** (Auto Loader, COPY INTO, Lakeflow Connect) gets data in reliably and idempotently, with checkpoints and provenance you can audit.

And the 2026 posture shift matters more than any individual feature: **managed Unity Catalog tables + liquid clustering + predictive optimization + schema enforcement with a `_rescued_data` escape hatch** is the new default floor. If your platform is still built on partitioned tables, nested mounts, manual `ZORDER`, and hand-scheduled `OPTIMIZE` jobs, you're not wrong — you're just paying for a layout decision the platform has already superseded.

At this level, being “advanced” doesn’t look like using more features. It looks like making **fewer, better-justified decisions** and writing them down.

And here’s the important part: everything you built at Level 2 is now waiting for logic. You have raw data landing faithfully, trusted conformed tables, and business marts. What you don’t yet have is a disciplined way to move data from one to the next — which transformations belong where, how to write SQL and PySpark that scales, and how to model slowly changing dimensions without painful rewrites.

Wait for the next detailed blog — Next Monday, 11 AM IST. We will go deep into the **Transformation Layer**: production SQL patterns, PySpark at scale, incremental processing with `MERGE`, deduplication strategies, SCD Type 1 and 2, dimensional modeling, and the transformation mistakes that get caught instantly in senior technical interviews.

Data Engineering + | Data Engineer + | Databricks + | Spark +

Big Data +